

Advanced Git

`git stash` when you want to record the current state of the working directory and the index, but want to go back to a clean working directory. The command saves your local modifications away and reverts the working directory to match the `HEAD` commit.

listed with `git stash list`,
inspected with `git stash show`
restored (potentially on top of a different commit) with `git stash apply`,

`git rebase` Transplant a series of commits onto a different starting point. You can also use `git rebase` to reorder or combine commits: see INTERACTIVE MODE below for how to do that. For example, imagine that you have been working on the `topic` branch in this history, and you want to "catch up" to the work done on the `master` branch.

If we merge commit from one branch to another

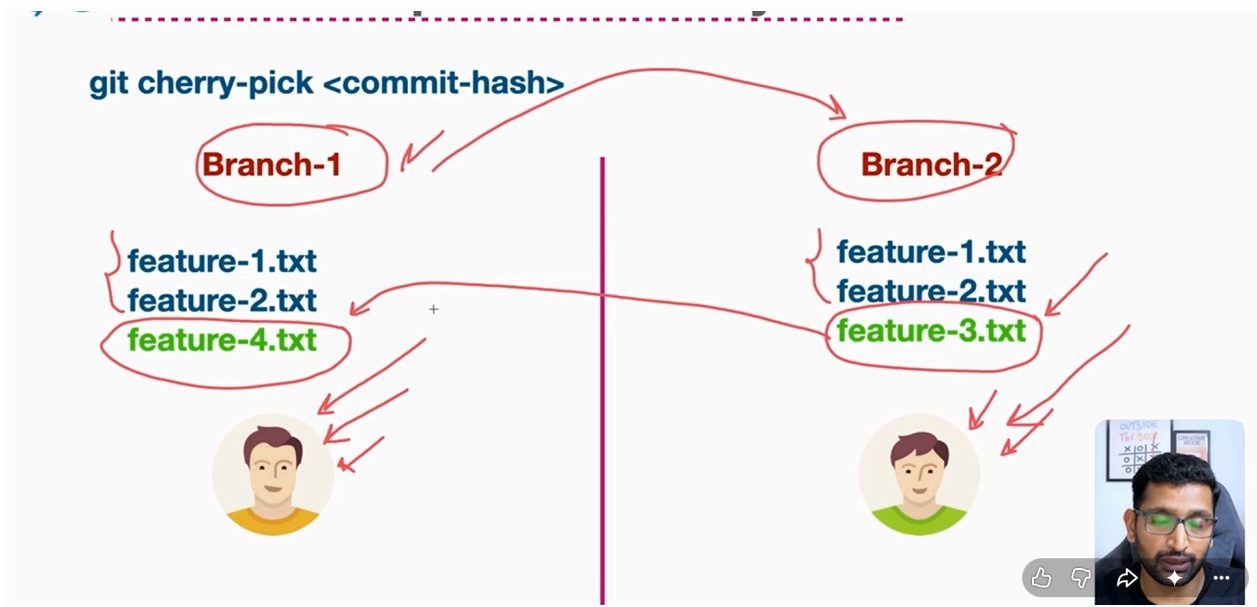
```
      A---B---C topic
      /
D---E---F---G master
```

If we merge commit from one branch to another using rebase to make clean commit history

```
E---F---G---H---I---J topicA
```

Git Cherry-Pick

The `git cherry-pick` command allows you to select **specific, individual commits** from one branch and apply them onto your current branch.



Git Reset

The git reset command moves your current branch (HEAD) to a different commit.

Depending on the option, it can also change which changes are staged or even delete changes from your working directory.

Use it to undo commits, unstage files, or clean up your history.

Summary of Git Reset Commands and Options

- `git reset --soft <commit>` - Move HEAD to commit, keep changes staged
- `git reset --mixed <commit>` - Move HEAD to commit, unstage changes (default)
- `git reset --hard <commit>` - Move HEAD to commit, discard all changes
- `git reset <file>` - Unstage a file
- `git log --oneline` - Show commit history

Git Revert

The `git revert` command undoes a previous commit by creating a new commit that reverses the changes.

This keeps your commit history intact and is the safest way to undo changes in a shared repository.

Summary of Git Revert Commands and Options

- `git revert HEAD` - Revert the latest commit
- `git revert <commit>` - Revert a specific commit
- `git revert HEAD~2` - Revert a commit further back in history
- `git revert --no-edit` - Skip commit message editor
- `git log --oneline` - Show commit history.

Git Tag

A **tag** in Git is like a label or bookmark for a specific commit.

Tags are most often used to mark important points in your project history, like releases (v1.0 or v2.0).

Tags are a simple and reliable way to keep track of versions and share them with your team or users.

Some common tag types include:

- **Releases:** Tags let you mark when your project is ready for release, so you (and others) can always find that exact version later.
- **Milestones:** Use tags to highlight major milestones, like when a big feature is finished or a bug is fixed.
- **Deployment:** Many deployment tools use tags to know which version of your code to deploy.

- **Hotfixes:** If you need to fix an old version, tags make it easy to check out and patch the right code.

Git Blame

Git blame is a Git command that annotates each line in a file with the following information.

- The last commit that modified the line.
- The author of the commit.
- The timestamp of the change.
- The line number and code content (optionally).